

Feature Brief: Accounts Receivable (AR) Module

For: Junior developer **Product:** E-Numerak (Django + Next.js) **Goal:** Give businesses a clear view of *who owes them money, how much, and how overdue* — built on top of the existing invoice + payment data.

1. What already exists (DO NOT rebuild)

The foundation is already in the codebase — build **on top** of it:

Thing	Where	Notes
<code>Payment</code> model	<code>apps/payments/models.py</code>	FK to Invoice; <code>amount</code> , <code>method</code> , <code>payment_date</code> , <code>reference</code> , <code>recorded_by</code>
<code>Invoice.amount_paid</code>	<code>apps/invoices/models.py</code>	Decimal — sum of payments
<code>Invoice.total_amount</code> , <code>due_date</code> , <code>currency</code>	<code>apps/invoices/models.py</code>	Already stored
Invoice statuses	<code>apps/common/constants.py</code>	<code>paid</code> , <code>partially_paid</code> already defined
<code>Customer</code>	<code>apps/customers/models.py</code>	AR is grouped per customer

So AR is mostly **computation + reporting + UI**, not new core data.

2. What to build

2.1 Outstanding balance (the core number)

Add a computed property on `Invoice`:

```
# apps/invoices/models.py (inside Invoice)
@property
def balance_due(self):
    """Amount still owed by the customer."""
    return (self.total_amount or 0) - (self.amount_paid or 0)

@property
def is_overdue(self):
    from django.utils import timezone
    return (
        self.balance_due > 0
        and self.due_date is not None
        and self.due_date < timezone.now().date()
        and self.status not in ('cancelled', 'deactivated', 'draft')
    )

@property
def days_overdue(self):
    from django.utils import timezone
    if not self.is_overdue:
        return 0
    return (timezone.now().date() - self.due_date).days
```

Only invoices that count as receivables: status is a *real, sent* invoice (not draft , cancelled , deactivated) and `balance_due > 0`.

2.2 Auto-update status when a payment is recorded

When a `Payment` is created/updated, recompute the invoice:

```
# apps/payments – after saving a Payment (signal or in the serializer/view)
def recompute_invoice_payment_state(invoice):
    paid = invoice.payments.aggregate(s=Sum('amount'))['s'] or 0
    invoice.amount_paid = paid
    if paid <= 0:
        pass # leave current status
    elif paid >= invoice.total_amount:
        invoice.status = 'paid'
    else:
        invoice.status = 'partially_paid'
    invoice.save(update_fields=['amount_paid', 'status'])
```

Use a Django **post_save / post_delete signal** on `Payment` so it always stays correct, even if a payment is deleted.

2.3 Aging buckets

Standard AR aging on the *overdue* part of each unpaid invoice:

Bucket	Meaning
Current	not yet due (<code>due_date >= today</code>)
1–30	1–30 days overdue
31–60	31–60 days overdue
61–90	61–90 days overdue
90+	more than 90 days overdue

A helper returns, per invoice, which bucket its `balance_due` falls into; the report sums balances per bucket (overall and per customer).

3. API endpoints (new)

Mount under `apps/reporting/` or a new `apps/receivables/` . All JWT-auth, scoped to the user's company.

Method	Path	Returns
GET	<code>/api/v1/reports/ar/summary/</code>	Totals: total receivable, total overdue, count, DSO
GET	<code>/api/v1/reports/ar/aging/</code>	Aging buckets (overall + optional <code>?customer_id=</code>)
GET	<code>/api/v1/reports/ar/by-customer/</code>	Per-customer outstanding + overdue (sortable)
GET	<code>/api/v1/reports/ar/customer/{id}/statement/</code>	Statement of account: invoices + payments + running balance
GET	<code>/api/v1/reports/ar/aging/export/</code>	CSV/Excel export of the aging report

Summary response example:

```
{
  "currency": "AED",
  "total_receivable": 152300.00,
  "total_overdue": 41200.00,
  "open_invoice_count": 18,
  "overdue_invoice_count": 5,
  "dso_days": 37
}
```

DSO (Days Sales Outstanding) = average days to collect. Formula: $(\text{total_receivable} / \text{total_credit_sales_in_period}) * \text{number_of_days_in_period}$. Start simple (last 90 days) — can refine later.

4. Frontend (Next.js)

New section "**Receivables**" in the app sidebar:

1. **AR Dashboard** — top cards (Total Receivable, Overdue, DSO), an aging bar chart, and a "Top debtors" list.
2. **Aging report table** — columns: Customer · Current · 1–30 · 31–60 · 61–90 · 90+ · Total. Filter by customer/date. Export button.
3. **Customer statement page** — all invoices + payments for one customer with a running balance; printable / PDF (reuse the existing PDF approach).
4. On the existing **invoice detail page**, show: Balance Due, Paid, Overdue badge, and a **"Record Payment"** action (creates a `Payment`).

Keep styling consistent with the existing dashboard components.

5. Optional (phase 2)

- **Payment reminders:** scheduled job that emails customers with overdue invoices (reuse `services/emails.py`). Configurable reminder days (e.g. on due date, +7, +30).
 - **Partial payment history** shown inline on the invoice.
 - **Credit notes** reduce the receivable (link existing credit-note flow to AR).
 - **Multi-currency** AR totals (group by `currency` ; do not mix).
-

6. Acceptance criteria (definition of done)

- [] Recording a payment updates `amount_paid` and flips status to `partially_paid` / `paid` automatically (and reverts if payment deleted).
 - [] `balance_due` , `is_overdue` , `days_overdue` correct on the invoice.
 - [] AR summary, aging, and by-customer endpoints return correct, company-scoped numbers (verified against a manual spreadsheet on test data).
 - [] Aging buckets add up to total receivable.
 - [] Customer statement shows a correct running balance.
 - [] Cancelled/draft/deactivated invoices are **excluded** from receivables.
 - [] Dashboard + aging table + statement pages render and export works.
 - [] Unit tests for balance/aging/status-recompute logic.
-

7. Suggested build order (small, safe steps)

1. Add `balance_due` / `is_overdue` / `days_overdue` properties + tests.
2. Add the Payment → invoice status recompute signal + tests.
3. Build the AR summary endpoint, then aging, then by-customer, then statement.
4. Frontend: dashboard → aging table → statement page → invoice "Record Payment".
5. (Phase 2) reminders + exports.

Start with steps 1–2 (pure backend logic, fully testable) before any UI.